

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 05

মেমরি ম্যানেজমেন্ট

নমস্কার পূর্ববর্তী ভিডিওতে আমরা দেখেছি যে, অপারেটিং সিস্টেমটি কীভাবে CPU পরিচালনা করতে পারে কারণ এটি সম্ভবত সিস্টেমে সবচেয়ে গুরুত্বপূর্ণ সম্পদ বা রিসোর্স। তর্কসাপেক্ষে, দ্বিতীয় সবচেয়ে গুরুত্বপূর্ণ সম্পদ, বরং সিস্টেমের দ্বিতীয় সবচেয়ে গুরুত্বপূর্ণ হার্ডওয়্যার রিসোর্স হল মেমরি। এই ভিডিওটিতে, আমরা দেখব কীভাবে অপারেটিং সিস্টেম মেমরি পরিচালনা করে? মূলত, আমরা দেখব কীভাবে অপারেটিং সিস্টেম সিস্টেমে উপস্থিত RAM-র পরিচালনা করে।

(স্লাইডটাইপ পড়ুন: 00:56)

তাহলে আসুন আমরা আবার এই বিশেষ স্লাইডটিকে পর্যালোচনা করি। আমরা দেখেছি যে যখন আমরা কোনও বিশেষ প্রোগ্রাম গ্রহণ করি যেটা হল যে কোনও প্রোগ্রাম যেকোনো ভাষায় লিপিবদ্ধ হতে পারে এবং যখন আমরা এটি কম্পাইল করি তখন আমরা এটিকে এক্সিকিউটেবল বলি। এখন, এক্সিকিউটেবলটি হার্ড ডিস্কে সংরক্ষণ করা হবে এবং যখনই কোনও ব্যবহারকারী এই প্রোগ্রাম চালায় বা এই প্রোগ্রামটি এক্সিকিউট করে, এটি একটি প্রসেস তৈরি করে। সুতরাং, এই প্রসেসটি অপারেটিং সিস্টেম দ্বারা তৈরি করা হয় এবং এটি RAM এ এক্সিকিউট হয়। তাই, অপারেটিং সিস্টেমটি মূলতঃ কি করবে যে এক্সিকিউটেবলটি হার্ডডিস্কের মধ্যে সঞ্চিত আছে, তা RAM এ লোড করা হবে এবং তারপর সেই নির্দিষ্ট প্রোগ্রামে এক্সিকিউশনটি প্রেরণ করা হবে এবং এই প্রোগ্রামটি তখন CPU এ এক্সিকিউট হবে। তাই, আমরা এই বিশেষ প্রসেসটি দেখেছি যা মেমরির বিভিন্ন সেগমেন্ট জুড়ে রয়েছে, যেমন টেক্সট সেগমেন্ট যেখানে এক্সিকিউটেবল নির্দেশাবলী থাকে, স্ট্যাক, হীপ এবং আরো অপারেটিং সিস্টেমের

কিছু গোপন মেটাডেটা যেমন রেজিস্টার, খোলা ফাইলের তালিকা এবং সম্পর্কিত প্রসেসগুলি। সুতরাং, এই সবগুলি মিলে আসলে প্রসেস গঠিত হয়। এখন, আমাদের জন্য গুরুত্বপূর্ণ হল এই প্রসেস এবং RAM-র মধ্যে এর উপস্থিতি।

(স্লাইডসময় পড়ুন: 02:45)

যেমন আমরা জানি RAM বা র‍্যান্ডম অ্যাকসেস মেমরি এছাড়াও যাকে প্রধান মেমরিও বলা হয় সেটি সিস্টেমের একটি সীমিত রিসোর্স। সুতরাং, প্রতিটি সিস্টেমে 4, 8, 16 বা 32 গিগাবাইট RAM থাকে। একই সময়ে আমাদের একাধিক প্রসেসগুলি প্রায় একযোগে এক্সিকিউট করতে পারে। সুতরাং, আমরা আগের ভিডিওতে যেমন দেখেছি, এই প্রসেসগুলি এক্সিকিউট হতে পারে টাইম স্লাইসড পদ্ধতিতে এবং যদি সিস্টেমে একাধিক প্রসেসর উপস্থিতি থাকে, তাহলে এই প্রসেসগুলি সমান্তরালেও এক্সিকিউট হতে পারে। যাইহোক, এই সমস্ত ক্ষেত্রে, এই প্রসেস প্রতি এবং তাদের সংশ্লিষ্ট মেমরি ম্যাপ RAM-এ থাকা উচিত। মূলত প্রসেস 1 এর মেমরি ম্যাপ, প্রসেস 2 এর মেমরি ম্যাপ, প্রসেস 3 এর মেমরি ম্যাপ এবং প্রসেস 4 এর মেমরি ম্যাপ র‍্যামে থাকা উচিত, যাতে এই 4 টি প্রসেস এক্সিকিউট হয় সমান্তরাল ভাবে বা একটি বহু টাস্ক বিশিষ্ট পরিবেশে। এখন, আমরা দেখতে যাচ্ছি কিভাবে অপারেটিং সিস্টেম এই রিসোর্সকে পরিচালনা করে বা অন্য কথায় RAM কিভাবে এটি একাধিক প্রসেসকে প্রায় একসাথে সিস্টেমে চালানোর অনুমতি দেবে।

(স্লাইডটাইপ পড়ুন: 04:25)

সুতরাং, অপারেটিং সিস্টেমে মেমরি পরিচালনা করার একটি সবচেয়ে পুরোনো উপায়টি সিংগল কন্টিগুওস মডেল হিসেবে পরিচিত। সুতরাং, মূলত এইখানে আমরা একটি RAM উপস্থিতি করেছি যা সিস্টেমের র‍্যাম এবং অপারেটিং সিস্টেম দ্বারা এটা নিশ্চিত করা হয়েছে যে এই RAM এ এক সময়ে একটিমাত্র প্রসেস থাকবে। মূলত কোনো নির্দিষ্ট তাত্ত্বিকভাবে একমাত্র প্রসেস এবং এটি মেমরির ম্যাপ যা RAM-র মধ্যে রয়েছে। সুতরাং, এই প্রসেসটির এক্সিকিউশন সমাপ্ত হওয়ার পর পরবর্তী প্রক্রিয়াকে RAM এ লোড করা হবে।

(স্লাইডটাইপ পড়ুন: 05:২1)

সুতরাং, দুর্বলতাগুলি স্পষ্ট।

(স্লাইডটাইপ পড়ুন: 05:25)

প্রথমত, আমরা ক্রমানুসারে এক্সিকিউশন কার্যকর করতে বাধ্য হই। যখন একটি প্রক্রিয়া সম্পূর্ণ হয়ে যায়, তখন কেবলমাত্র দ্বিতীয় প্রসেসটি RAM দখল করে। এই বিশেষ মডেলের আরেকটি সীমাবদ্ধতা হল একটি সিংগল কন্টিগুওস মডেল অর্থাৎ প্রসেসটির আকারটি RAM এর আকারের দ্বারা সীমাবদ্ধ। উদাহরণস্বরূপ, আসুন

আমরা বলি যে আমাদের কাছে একটি র‍্যাম আছে যা 12 কিলো বাইটের মত, যখন এটি খুব ছোট পরিমাণের RAM বলে মনে হয়, এমবেডেড সিস্টেমে এই আকারের RAM প্রচলিত। সুতরাং, এই র‍্যামটি 12 কিলো বাইট ধরি এবং আমাদের বলা হল যে আমাদের প্রসেসরের আকারটি 100 কিলো বাইটের। তারপর এটি বেশ স্পষ্ট যে এই প্রক্রিয়া এই RAM এ ব্যবহার করে চালানো যাবে না। মূলতঃ RAMটি সম্পূর্ণ প্রসেসরটি রাখার জন্য যথেষ্ট নয়।

(স্লাইডটাইপ পড়ুন: 06:35)

পরবর্তী মডেল যেটা আমরা দেখব, যা সিংগল কন্টিগুওস মডেলের উপর সামান্য উন্নতি হয় যা একটি পার্টিশন মডেল হিসাবে পরিচিত। মূলত এই মডেলে যেকোনও মুহূর্তে, আমরা একাধিক প্রসেস করতে পারি যা র‍্যামকে একই সময়ে দখল করে। উদাহরণস্বরূপ, এই বিশেষ ক্ষেত্রে আমাদের দুটি প্রসেস আছে। এই নীল প্রক্রিয়া এবং সবুজ প্রক্রিয়া যা র‍্যামকে ধরে রাখে এবং সেইজন্য, প্রসেসরটি এই প্রসেসকে এক্সিকিউট করে পাশাপাশি এই প্রসেস সমান্তরালভাবে বা টাইম স্লাইসড পদ্ধতিতেও এক্সিকিউট করতে পারে। এখন, এই ধরনের পার্টিশন পরিচালনা করার জন্য, অপারেটিং সিস্টেমের জন্য পার্টিশন টেবিল নামে পরিচিত কিছু প্রয়োজন হবে। সাধারণত এই পার্টিশন টেবিল উপস্থিত RAM-র মধ্যেও উপস্থিত থাকবে। এটি এমন একটি এলাকায় উপস্থিত থাকবে যা এখানে দেখানো হয় না। সুতরাং, পার্টিশন টেবিলে একটি প্রসেসরের বেস অ্যাড্রেস, প্রসেসরের আকার এবং একটি প্রসেস শনাক্তকারী থাকবে। উদাহরণস্বরূপ, নীল প্রসেসরের মেমোরি বা 0x0 এর একটি বেস অ্যাড্রেস ইঙ্গিত করে যে ওটা র‍্যামের শূন্য অবস্থান থেকে শুরু হয় এবং এই প্রসেসরের 120 কিলো বাইটের আকার রয়েছে। সুতরাং, 120 কিলো বাইট মানে এই বিন্দু পর্যন্ত। এখানে একটি পতাকাও রয়েছে যা ইউসেজ ক্ল্যাগ নামেও পরিচিত, যা উল্লেখ করে যে র‍্যামের এই নির্দিষ্ট এলাকাটি ব্যবহৃত হচ্ছে না ফাকা আছে। উদাহরণস্বরূপ, আসুন আমরা এই প্রসেসটিকে প্রসেস 1 ধরছি, এটি এখানে দেখানো হয়েছে যে এখানে সবুজ 1। সুতরাং, এই প্রক্রিয়াটি একটি মেমরি অবস্থান 120K শুরু হয় অর্থাৎ এই বিন্দু এবং এর 60K একটি আকার আছে। সুতরাং, এটি 180K পর্যন্ত প্রসারিত এবং এই এলাকাটিও ব্যবহৃত হচ্ছে। এখন, পার্টিশন টেবিলে আরেকটি এন্ট্রি আছে যা ফ্রি হিসাবে নির্দিষ্ট। সুতরাং, এটি 180K থেকে শুরু হয় এবং একটি 30K আকারের জন্য প্রসারিত। সুতরাং, এখানে এই সাদা এলাকার অনুরূপ। সুতরাং, অপারেটিং সিস্টেম সম্ভবত এই ফ্রি মেমরিটি সম্ভবত তৃতীয় প্রসেসটি চালানোর জন্য ব্যবহার করতে পারে এবং সেইজন্য, একই সময়ে RAM-তে উপস্থিত তিনটি প্রসেস থাকতে পারে।

(স্লাইডটাইপ পড়ুন: 09:34)

তাই ধরা যাক একটি নতুন প্রসেস আইডেনটিফায়ার 5 দ্বারা এইমাত্র তৈরি হচ্ছে। সুতরাং, অপারেটিং সিস্টেমটি পার্টিশন টেবিলে দেখবে এবং এই নতুন প্রসেসের জন্য একটি নির্দিষ্ট এন্ট্রি তৈরি করবে। এখন, যেহেতু এই

প্রসেসটির জন্য ২০ কিলো বাইটের RAM প্রয়োজন তাই অপারেটিং সিস্টেমটি এই প্রক্রিয়ার জন্য ২০ কিলো বাইট র‍্যাম বরাদ্দ করবে। সুতরাং, এই বরাদ্দ মুক্ত স্থান থেকে করা হয় এবং আমরা এখানে দেখি, তৃতীয় প্রসেসটি এখন RAM-র মধ্যে উপস্থিত যেখানে ফ্রী স্পেসটির আয়তন হ্রাস হয়। সুতরাং, আমাদের এখন ১০ কিলো বাইট মুক্ত স্থান আছে। অনুরূপভাবে, যখন একটি প্রসেস এক্সিকিউশন সমাপ্ত হয়, উদাহরণস্বরূপ, যখন প্রসেস ১ তার কার্যকারিতা সম্পূর্ণ করে, তখন যে পরিমাণ RAM থাকে তা ডি-অ্যালোকট করা হবে। ফলস্বরূপ বিভাজন টেবিলে সংশ্লিষ্ট এন্ট্রিটি ফ্রী হবে। সুতরাং, RAM টি ৬০K মুক্ত মেমরি হবে। তাই, এটি এই নির্দিষ্ট এলাকায় র‍্যামের সাথে মিলিত হয় যা প্রসেস ১ দ্বারা ব্যবহৃত হয়। যদিও এই র‍্যামের সাথে ভাগ করা পদ্ধতিটির ফলে, RAM-র মধ্যে একাধিক প্রসেস বরাদ্দ করা সম্ভব হয়। এটি ফ্র্যাগমেন্টেশন নামে পরিচিত।

(স্লাইডটাইপ পড়ুন: ১১:১৮)

সুতরাং, আসুন আমরা দেখি যে কিভাবে আমাদের RAM টি দেখতে হয় এবং আমরা দেখেছি যে আমাদের মোট ৭০ কিলো বাইট অব্যবহৃত র‍্যাম আছে অর্থাৎ আমাদের ৬০ কিলো বাইট রয়েছে সঙ্গে আরও ১০। সুতরাং, এটি অব্যবহৃত ৭০ কিলো বাইট র‍্যাম। যাইহোক, ৭০ কিলোবাইট র‍্যাম থাকা সত্ত্বেও, একটি নতুন প্রসেস যেমন আইডি ৬ প্রসেস, যার আকার ৬৫ k শুরু করা যাবে না যদিও ৭০ কিলোবাইট ফ্রী RAM পাওয়া যায় এবং এটির কারণ ৭০ কিলোবাইট মেমরির কিলো বাইটগুলি সংলগ্ন অবস্থানে নেই। উদাহরণস্বরূপ, এখানে ৬০ কিলো বাইট মুক্ত মেমরি আছে এবং সেইসঙ্গে ১০ মেগাবাইট মুক্ত মেমরিও রয়েছে। সুতরাং, স্বতন্ত্রভাবে এই মুক্ত ব্লকের প্রতিটি ব্লক ৬৫ কিলো বাইটের একটি নতুন প্রসেস শুরু করার জন্য যথেষ্ট নয়। আর এটিই হল ফ্র্যাগমেন্টেশন যার ফলে RAM এর আন্ডার ইউটিলাইজেশন হয়।

(স্লাইডটাইম পড়ুন: ১২:৩৭)

এখন, আসুন আমরা ধারণা করি যে যখন একটি নতুন প্রসেস আসে তখন RAM মেমরিতে প্রচুর পরিমাণে ফাঁকা জায়গা উপস্থিত থাকে। সুতরাং, এর পর অপারেটিং সিস্টেমটিকে এই সিদ্ধান্ত নিতে হবে যে এই মুক্ত মেমরিগুলির মধ্যে কোথায় নতুন প্রক্রিয়াটি লোড করা উচিত। উদাহরণস্বরূপ এখানে, এখানে একটি নতুন প্রক্রিয়া আছে যা এইমাত্র এসেছে এবং আমাদের কাছে ৩ টি ব্লক মুক্ত মেমরি আছে, এই এক, এই এক এবং এই এক। সুতরাং, এই তিনটি ব্লক বা মুক্ত মেমরির RAM এ নতুন প্রসেসটি কোনটিতে লোড হবে? এটার জন্য অনেক আলগোরিদম হতে পারে। সুতরাং, সবচেয়ে সহজ অ্যালগরিদমটি ফার্স্ট ফিট হিসাবে পরিচিত। সুতরাং, এই ফার্স্ট ফিট অ্যালগরিদমে অপারেটিং সিস্টেমটি কি করবে তা হল এই যে উপরের দিকে থেকে শুরু করে মুক্ত ব্লকগুলি স্ক্যান করে এবং প্রথম উপলভ্য ফ্রী ব্লকটি ব্যবহার করে যা প্রসেসের চাহিদার হিসাবে যথেষ্ট বড়। উদাহরণস্বরূপ, যদি এই RAM উপরে থেকে স্ক্যান করা হয় আর এটা দেখা যায় যে প্রথম মুক্ত ব্লকটি প্রসেসটির জন্য যথেষ্ট মাপসই নয়। তবে, এটি এখানে বরাদ্দ করা হয় না। পরবর্তী মুক্ত ব্লক এই প্রসেসকে বরাদ্দ করতে যথেষ্ট বড়। অতএব, প্রসেসটি

এখানে বরাদ্দ করা হয়। এই ফাস্ট ফিট অ্যালগরিদম যখন একটি OS এর দৃষ্টিকোণ থেকে সঞ্চালন করা অত্যন্ত সহজ, এটি ফ্র্যাগমেন্টেশন খারাপ করতে পারে। মূলতঃ প্রথমত, আমরা যা করছি তা হলো আমরা বড় মুক্ত ব্লকটিকে ছোট ছোট অংশে ভাগ করছি। এইভাবে পৃথকভাবে প্রতিটি অংশ একটি একক প্রসেস পেতে সক্ষম হবে না, এইভাবে ফ্র্যাগমেন্টেশন খারাপ হয়। অন্য এলগরিদম যা আমরা দেখতে পাব বেস্ট ফিট অ্যালগরিদম হিসাবে পরিচিত এবং এই ফ্র্যাগমেন্টেশন সম্পর্কে খুব ভাল। মূলত এটিতে ফাস্ট ফিট অ্যালগরিদমের মত মেমরি ফ্র্যাগমেন্ট হবে না। সুতরাং, এখানে কি ঘটে তা হল যখন একটি নতুন প্রসেস প্রবেশ বা তৈরি করা হয়, অপারেটিং সিস্টেমটি যে সমস্ত মুক্ত ব্লকগুলি পাওয়া যায় তা স্ক্যান করবে এবং সেই ব্লকটি নির্বাচন করবে যা ওই প্রসেসের জন্য সবচেয়ে উপযুক্ত। সুতরাং, এই বিশেষ ক্ষেত্রে যেমন আমরা আগের মতো দেখেছি, এই নতুন প্রসেসটি পূরণ করার জন্য এই বিশেষ ফ্রী ব্লক খুবই ছোট। এই ফ্রী ব্লক খুব বড় যখন তৃতীয় মুক্ত ব্লকটি সঠিক। সুতরাং, অপারেটিং সিস্টেম এই প্রসেসটিকে এই মুক্ত ব্লকের মধ্যে বরাদ্দ করবে যেমন দেখানো হয়েছে এখানে। বেস্ট ফিট অ্যালগরিদমটি যথাযথভাবে RAM-র ব্যবহারে ফ্র্যাগমেন্টেশন সমস্যা হ্রাস করার সময় একটি পারফরম্যান্স হিট হতে পারে। মূলত, এটি পারফরম্যান্সের অবনতি ঘটতে পারে। কারণ হচ্ছে যে এখন অপারেটিং সিস্টেমটি প্রত্যেকটি মুক্ত ব্লকের মাধ্যমে স্ক্যান করতে হবে এবং এটি একটি নতুন সিদ্ধান্তের জন্য সবচেয়ে ভাল ব্লক সম্পর্কে সিদ্ধান্ত নেওয়ার প্রয়োজন এবং এর ফলে কিছুটা সময় লাগবে।

(স্লাইডটাইপ পড়ুন: 16:২৪)

পার্টিশন মডেলের আরেকটি সমস্যা হল ডিঅ্যালোকেশন(deallocation)। সুতরাং, মূলত ডিঅ্যালোকেশন ঘটে যখন একটি প্রসেস তার এক্সিকিউশন সম্পন্ন করে বা ওটা সমাপ্ত হয় এবং একটি নতুন প্রসেস বা অন্য কোনও প্রসেস RAM থেকে চালানোর অনুমোদন দেবার জন্য RAM থেকে মুছে ফেলা হবে। সুতরাং, ডিঅ্যালোকেশনের জন্য প্রয়োজন নতুন অর্থাৎ নতুন একটি ফ্রি ব্লক তৈরি হবে এবং এর ফলে পার্টিশন টেবিলে এই ব্লকের অনুরূপ ফ্রি ফ্ল্যাগ সেট হবে। সুতরাং, অপারেটিং সিস্টেমটিকে কি করতে হবে তা হল এটিকে সনাক্ত করতে হবে যে RAM এ প্রকৃতপক্ষে তিনটি সংলগ্ন মুক্ত ব্লক রয়েছে এবং এটি এই তিনটি ঘনিষ্ঠ ফ্রি ব্লক সনাক্ত করে এইগুলিকে একত্রিত করে একটিমাত্র ব্লক করতে সক্ষম হবে।

(স্লাইডসময় পড়ুন: 17:34)

এটি একটি বড় মুক্ত ব্লকের মধ্যে মার্জ করার সুবিধা যে এখন এই বড় ফ্রী ব্লক একটি বৃহত্তর প্রসেস পূরণ করতে পারে এবং যার ফলে ফ্র্যাগমেন্টেশনের প্রভাব হ্রাস পায়।

(স্লাইডটাইপ পড়ুন: 17:51)

সুতরাং, আমরা দেখেছি যে সিংগল কন্টিগুওস মডেলের পাশাপাশি পারটিশন মডেলের প্রধান সীমাবদ্ধতাটি হল সম্পূর্ণ এক্সিকিউসন চলাকালীন সম্পূর্ণ মেমরির ম্যাপ RAM- এ উপস্থিত থাকা প্রয়োজন। সুতরাং, সম্পূর্ণ প্রসেসের জন্য সমস্ত বরাদ্দ কন্টিগুওস মেমোরিতে থাকা প্রয়োজন এবং এই সমস্যাগুলির কারণে, বেসের জন্য প্রসেসটির আকার RAM এর আকারের উপর নির্ভর করে, এবং কর্মক্ষমতা হ্রাস পায় বই সংরক্ষণের কারণে আর পারটিশনগুলির ব্যবস্থাপনাতেও। সুতরাং, সৌভাগ্যবশত আমাদের জন্য আধুনিক দিনের অপারেটিং সিস্টেম মেমরি পরিচালনার ক্ষেত্রে অনেক ভালো। অধিকাংশ আধুনিক দিনের অপারেটিং সিস্টেম দুটি ধারণা ব্যবহার করে যা ভার্চুয়াল মেমরি এবং সেগমেন্টেশন। তাই পরবর্তী কয়েকটি ভিডিওতে, আমরা এই মেমরি ধারণার পরিবর্তে এই মেমরি ব্যবস্থাপনা ধারণাগুলি দেখব এবং আমরা দেখতে পাব কিভাবে Intel x86 প্রসেসর মেমরি পরিচালনা করে। ধন্যবাদ.